SERIALIZATION CHOICES

YAML for Configuration Files

YAML is appropriate for sensor configuration because it is highly human-readable and easy to edit manually. Farmers or administrators may need to change sensor IDs, reporting intervals, or network settings without modifying source code. YAML uses minimal syntax compared to XML or JSON, making configuration files cleaner and less verbose. Although YAML is slower to parse than JSON or Protobuf, configuration files are read infrequently, so performance is not a major concern.

Protobuf for Sensor-to-Server Communication

Protobuf are ideal for the sensor to server communication because they provide high performance (through binary encoding) and low verbosity. Sensor systems continuously transmit readings, so minimizing bandwidth and processing overhead is important. Human readability is sacrificed because Protobuf data is binary, which is necessary, since machines are what process the telemetry stream.

JSON/XML for the Public REST API

The REST API uses JSON and XML because public client applications require interoperable, widely supported formats.

JSON is preferred for modern applications because it is **lightweight, less verbose than XML, easy to parse in web browsers and mobile apps and highly human-readable.** This makes it suitable for the mobile dashboard and WebSocket messages.

XML is included because the legacy desktop client only supports XML. XML is more verbose than JSON but provides stronger document structure and mature schema technologies.

Using JSON/XML through HTTP content negotiation allows the same API endpoint to support different client capabilities without changing the backend logic.

WEB SERVICES PROTOCOL

REST is well suited for the historical-data API because the system naturally exposes data as resources that clients can retrieve, create, or delete using standard HTTP methods. A client requesting historical telemetry data is essentially asking for representations of resources such as sensor readings over HTTP.

The statelessness of REST improves scalability and reliability as the server does not need to remember previous requests to process a new one and

avoids maintaining heavy session state, allowing many farmers to access the system concurrently.

Unlike REST, SOAP is operation oriented rather than resource oriented and therefore difficult to expose data.

Just as with SOAP, RPC is action oriented and not resource oriented as it focuses on invoking functions remotely therefore difficult to expose the data.

ASYNCHIO VS THREADING

The system is primarily an I/O-bound problem because most of the program's time is spent waiting for external operations like waiting for sensor data rather than performing heavy CPU computation.

Coroutines are preferred because they use less resources compared to threads where each thread would require its own stack, OS scheduling and context switching. Therefore, a large number of sensor connections could create unnecessary overhead. In this case coroutines are much lighter because they run in user space under one event loop.

Threads are more suitable when background tasks must run independently and processes are preferable for CPU bound workloads for example AI inference for data interpretation.

HTTP MECHANICS

Content negotiation

Content negotiation directly affects the REST API design because the system must support multiple client types that expect different response formats. The server therefore examines the HTTP "Accept" header to determine which representation to return. The same endpoint returns the same underlying data but serialized differently.

This affects the design because now the server must implement multiple serializers, API handlers should separate internal data structures from output formatting and appropriate Content-Type headers must be returned.

WebSocket Upgrade Handshake

The WebSocket upgrade handshake is important because the live telemetry feed starts as a normal HTTP request and then transitions into a persistent full-duplex connection.

This strongly affects the system design as it will need a connection manager for active WebSocket clients, subscription tracking for sensor-specific updates and asynchronous broadcasting using AsyncIO coroutines.